

Algorithms Programming Project II

Dynamic Programming

Team members

Divya Upadhyay - 29228848

Contributions - Worked on problem 1 and 3 and comparative study

Raj Vora - 35551411

Contributions - Prepared report, worked on problem 2 and comparative study

Design and Analysis of Algorithms

Algorithm 1

Algorithm

1. Initialize two variables 'max_size' and 'boundary' to 0 and an empty list, respectively.
2. Loop through each element in the matrix using two nested for loops.
3. If the current element is greater than or equal to the threshold value 'h', then find the largest square matrix that can be formed with this element as the top-left corner.
4. To find the largest square matrix, loop through all possible sizes of the matrix (starting from 1 and going up to the minimum of the remaining rows and columns).
5. For each size, check if all the elements in the matrix of that size (starting from the current element) are greater than or equal to 'h'.
6. If all the elements in the matrix of that size are greater than or equal to 'h', then update the 'max_size' variable and the 'boundary' list to store the size of the largest square matrix and its boundaries, respectively.
7. Return the 'boundary' list, which contains the boundaries of the largest square matrix.

Correctness Proof

Proof by induction

Base case:

When the size of the square matrix is 1, the algorithm checks if the single element is greater than or equal to the threshold value 'h'. If it is, then it returns the boundary of this single-element square matrix, which is correct. Hence, the base case holds.

Inductive hypothesis:

Assume that the algorithm correctly finds the largest square matrix with minimum element value greater than or equal to 'h' for all sizes less than or equal to 'k'.

Inductive step:

Consider a square matrix of size 'k+1'. The algorithm finds the largest square matrix with minimum element value greater than or equal to 'h' with the top-left corner at some element 'e'. Let the size of this square matrix be 's'.

Case 1: If the top row or leftmost column of the square matrix is cut off, then the remaining sub-matrix is also a square matrix with minimum element value greater than or equal to 'h'. By the inductive hypothesis, the algorithm correctly finds the largest square matrix with minimum element value greater than or equal to 'h' for this sub-matrix.

Case 2: If the top row and leftmost column of the square matrix are not cut off, then the algorithm checks if all the elements in this square matrix have a value greater than or equal to 'h'. If they do, then the algorithm returns the boundaries of this square matrix, which is correct. If not, then the largest square matrix with minimum element value greater than or equal to 'h' must be contained within this square matrix, but with top-left corner at some element other than 'e'. Since this sub-matrix has a size less than or equal to 'k', by the inductive hypothesis, the algorithm correctly finds the largest square matrix with minimum element value greater than or equal to 'h' for this sub-matrix.

Therefore, the algorithm correctly finds the largest square matrix with minimum element value greater than or equal to 'h' for all square matrices of size less than or equal to 'k+1'. By induction, the algorithm is correct for all square matrices.

Time Complexity Analysis

The algorithm has three nested loops, and each loop iterates up to the length of one side of the input matrix. Therefore, the time complexity of the algorithm is proportional to m^3n^3 .

To see why, consider the outermost loop iterating over rows, which has m iterations. For each row, the algorithm loops over columns, which also has n iterations. Then, for each possible starting position of a square matrix (i,j) , the algorithm iterates over all possible sizes of square matrices, up to the minimum of the remaining rows and columns ($m-i$ and $n-j$). This results in a nested loop with up to $m*n$ iterations.

Thus, the total number of iterations of the three nested loops is $mn(m-i)(n-j)$ for each possible starting position (i,j) , and the worst-case scenario is when all the values in the input matrix are greater than or equal to 'h'. This gives the time complexity of $\Theta(m^3n^3)$.

Space Complexity analysis

The algorithm does not store any additional data structures or use any recursive function calls that would require additional memory. Instead, it works by iterating through the elements of the input matrix and updating the boundaries of the largest square matrix found so far.

Therefore, the only space used by the algorithm is the memory required to store the input matrix itself, which is typically assumed to be already present in memory and not counted towards the space complexity of the algorithm.

Hence, the space complexity of the brute-force algorithm is $\Theta(1)$.

Algorithm 2

Algorithm

The algorithm starts by initializing `max_size` to 0 and `boundary` to an empty list. Then it loops through all the elements in the matrix using two nested for-loops.

For each element in the matrix, if its value is greater than or equal to `h`, the algorithm considers it as the top-left corner of a potential maximal square. It then checks whether the square can be extended to the right and downwards, by checking the values of the elements in the next rows and columns. If all the elements satisfy the condition that their values are greater than or equal to `h`, the square size is increased by 1 and the loop continues.

If any element fails to satisfy the condition, `is_square` is set to `False`, and the loop breaks. If the size of the square obtained is greater than the previous maximum size, the boundary of the maximal square is updated.

Finally, the algorithm returns the boundary of the maximal square in the form of a list containing the coordinates of the top-left and bottom-right corners of the square.

Correctness Proof

proof by induction.

Base case:

When the input matrix has only one element, the algorithm correctly finds the maximal square with height greater than or equal to `h`.

Induction hypothesis:

Assume that the algorithm correctly finds the maximal square with height greater than or equal to `h` for any input matrix of size $i \times j$, where $1 \leq i \leq m$ and $1 \leq j \leq n$.

Induction step:

Consider an input matrix of size $(m+1) \times (n+1)$. The algorithm starts by initializing `max_size` to 0 and `boundary` to an empty list. It then loops through all the elements in the matrix using two nested for-loops.

For each element in the matrix, if its value is greater than or equal to `h`, the algorithm considers it as the top-left corner of a potential maximal square. It then checks whether the square can be extended to the right and downwards, by checking the values of the elements in the next rows and columns. If all the elements satisfy the condition that their values are greater than or equal to `h`, the square size is increased by 1 and the loop continues.

If any element fails to satisfy the condition, `is_square` is set to `False`, and the loop breaks. If the size of the square obtained is greater than the previous maximum size, the boundary of the maximal square is updated.

To prove the correctness of the algorithm, we need to show that it correctly finds the maximal square with height greater than or equal to `h` in the input matrix of size $(m+1) \times (n+1)$.

Case 1: The maximal square with height greater than or equal to h has its top-left corner outside the $(m+1) \times (n+1)$ submatrix.

In this case, the algorithm correctly initializes `max_size` to 0 and `boundary` to an empty list and continues the loop. Since the top-left corner of the maximal square is outside the $(m+1) \times (n+1)$ submatrix, the algorithm does not update the value of `max_size` or `boundary`. Therefore, the algorithm correctly finds that there is no maximal square with height greater than or equal to h in the $(m+1) \times (n+1)$ submatrix.

Case 2: The maximal square with height greater than or equal to h has its top-left corner inside the $(m+1) \times (n+1)$ submatrix.

In this case, the algorithm correctly initializes `max_size` to 0 and `boundary` to an empty list and loops through all the elements in the $(m+1) \times (n+1)$ submatrix. For each element in the submatrix with a value greater than or equal to h , the algorithm considers it as the top-left corner of a potential maximal square and checks whether the square can be extended to the right and downwards.

If all the elements satisfy the condition that their values are greater than or equal to h , the square size is increased by 1 and the loop continues. The algorithm repeats this process until it finds the maximal square with height greater than or equal to h in the $(m+1) \times (n+1)$ submatrix.

Therefore, by induction, the algorithm correctly finds the maximal square with height greater than or equal to h for any input matrix of size $m \times n$.

Time Complexity analysis

The algorithm uses two nested for-loops to iterate over all the elements in the input matrix. Hence, the time complexity of the outer loop is $O(m)$ and the time complexity of the inner loop is $O(n)$. Therefore, the time complexity of the nested loops is $O(m * n)$.

Inside the nested loops, the algorithm uses another loop to check whether a square can be extended to the right and downwards. The worst-case scenario occurs when the entire matrix consists of elements with values greater than or equal to h , and in this case, the third loop would iterate over a maximum of $(m-1) * (n-1)$ elements. Therefore, the time complexity of the third loop is $O(m * n)$.

Overall, the time complexity of the algorithm is the product of the time complexity of the nested loops and the time complexity of the third loop, i.e., $\Theta((m * n) * (m * n)) = \Theta(m^2 * n^2)$.

Therefore, the given algorithm has a time complexity of $\Theta(m^2 * n^2)$ in the worst case.

Space Complexity analysis

The algorithm declares only five integer variables - `max_size`, `i`, `j`, `size`, and `k` - and an empty list `boundary`. These variables store integer values and do not depend on the size of the input matrix. Hence, the space complexity of the algorithm is $O(1)$.

Furthermore, the input matrix is read from standard input and stored in a list called matrix. However, the input matrix is already present in the input and needs to be stored for processing. Hence, the space complexity of the input is not counted towards the space complexity of the algorithm.

Therefore, the space complexity of the given algorithm is $O(1)$.

Algorithm 3

Algorithm

Definition: $dp[i][j]$ = length of the largest square submatrix ending at $M[i][j]$ with all elements greater than or equal to h

Goal: $\max(dp[i][j])$ over all i and j

Cases:

For each (i, j) position, the algorithm updates $dp[j]$ based on the values of $dp[j-1]$ (left), $dp[j]$ (top), and $prev$ (top-left), where $prev$ is the value of $dp[j]$ before the update. If $dp[j]$ is greater than the current maximum size of the square submatrix, the maximum size and the corresponding boundary are updated.

Bellman equation:

$$dp[i][j] = \begin{cases} 0 & \text{if } M[i][j] < h \\ 1 & \text{if } i = 0 \text{ or } j = 0 \\ 1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) & \text{if } M[i][j] \geq h \end{cases}$$

Correctness Proof

Base case: For a square submatrix of length 1, the algorithm correctly sets $dp[i][j] = 1$ if $M[i][j] \geq h$ and $dp[i][j] = 0$ otherwise. Therefore, the base case is correct.

Inductive hypothesis: Assume that the algorithm correctly computes $dp[i][j]$ for all square submatrices of length less than or equal to k , where $k \geq 1$.

Inductive step: Consider a square submatrix of length $k+1$ with the top-left corner at (i, j) . We need to show that the algorithm correctly computes $dp[i][j]$.

If $M[i][j] < h$, then the square submatrix cannot include (i, j) and therefore $dp[i][j] = 0$, which is correctly computed by the algorithm.

If $M[i][j] \geq h$, then the square submatrix includes (i, j) . Let len be the length of the largest square submatrix with all elements greater than or equal to h ending at (i, j) . By definition, we have $dp[i][j] = len$.

If $len \leq k$, then the algorithm correctly computes $dp[i][j]$ by the inductive hypothesis.

If $len = k+1$, then the square submatrix must be of size $(k+1) \times (k+1)$. This means that the top, left, and top-left cells of the square submatrix must also have values greater than or equal to h . Therefore, we have $dp[i][j] = \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1$. This is correctly computed by the algorithm since it uses the values of $dp[i-1][j]$, $dp[i][j-1]$, and $dp[i-1][j-1]$, which are all correct by the inductive hypothesis.

Therefore, by induction, the algorithm correctly computes $dp[i][j]$ for all square submatrices of length less than or equal to $k+1$. This completes the proof of correctness.

Time Complexity Analysis

The algorithm loops over each cell in the input matrix once, so the outer loop runs m times and the inner loop runs n times, resulting in a total of mn iterations.

For each cell, the algorithm performs a constant amount of work, including comparisons, assignments, and updates to the dp array, and updating the max_size and boundary variables if a larger square submatrix is found. Therefore, the time complexity of the algorithm is proportional to the size of the input matrix, which is $\Theta(mn)$.

In practice, the algorithm may be faster or slower depending on the specific input data and the efficiency of the implementation, but its worst-case time complexity is $\Theta(mn)$.

Space Complexity analysis

The algorithm maintains an array dp of length n to store the lengths of the largest square submatrices ending at each column in the current row. The algorithm only needs to store the values for the current row and the previous row to compute the values for the next row. Therefore, the space complexity of the algorithm is $O(2n) = O(n)$.

In addition, the algorithm maintains variables max_size and $boundary$ to keep track of the largest square submatrix found so far and its coordinates, which take a constant amount of space.

Overall, the space complexity of the algorithm is $O(n)$, which is proportional to the number of columns in the input matrix.

Algorithm 4

Algorithm

Definition: $dp[i][j]$ = length of the largest square submatrix ending at $M[i][j]$ with all elements greater than or equal to h . Corners can be less than h

Goal: $\max(dp[i][j])$ over all i and j

Cases:

$DP(i, j) = k$, if (i, j) is the top left corner of a square of side length k and all elements in the square are greater than or equal to ' h '. We check special cases for the corners such that can be less than h

$DP(i, j) = 0$, if (i, j) is not the top left corner of any square with values greater than h and corners as any value.

Bellman Equation:

$$= k - j + 1 \quad \text{if } p(i, k) \geq h \text{ and } p(i, j) < h \text{ and } k == j+1$$

$$DP(i, k) = k - j \quad \text{if } p(i, k) \geq h \text{ and } p(i, j) \geq h$$

$$= DP(i, k-1) \quad \text{if and } p(i, k) < h$$

Correctness Proof

Base Case:

For a 1×1 matrix, the algorithm returns the correct answer since there is only one element in the matrix, and it is either greater than or equal to ' h ' or not.

Induction Hypothesis:

Assume that the algorithm is correct for a binary matrix of size $(m-1) \times (n-1)$.

Induction Step:

We will prove that the algorithm is correct for a binary matrix of size $m \times n$.

Let $DP(i, j)$ be the maximum length of the square-shaped area with (i, j) as the top left corner of the square. The algorithm fills the DP matrix using the bellman equation and finds the largest square-shaped area in the matrix.

We need to prove two things:

The algorithm correctly fills the DP matrix for all (i, j) pairs.

The algorithm correctly finds the largest square-shaped area in the matrix.

Proof of statement 1:

Consider a pair (i, j) in the matrix.

If $p(i, j) < h$, then $DP(i, j)$ must be 0. The algorithm sets $DP(i, j) = DP(i, j-1)$ since (i, j) cannot be the top left corner of any square-shaped area with all elements greater than or equal to 'h'.

If $p(i, j) \geq h$, then $DP(i, j)$ can be computed using the bellman equation. The algorithm correctly sets $DP(i, j)$ to the maximum length of the square-shaped area with (i, j) as the top left corner of the square.

Thus, the algorithm correctly fills the DP matrix for all (i, j) pairs.

Proof of statement 2:

Let the maximum size of the square-shaped area be 'k'. The algorithm iteratively computes the DP values and updates the value of 'max_size' to be the maximum value of $DP(i, j)$ encountered so far.

Consider a pair (i, j) such that $DP(i, j) = k$. The algorithm checks if all elements in the square-shaped area with (i, j) as the top left corner of the square are greater than or equal to 'h'. If this condition is satisfied, then the algorithm returns the bounding indices of the largest square-shaped area.

If all elements in the square-shaped area with (i, j) as the top left corner of the square are not greater than or equal to 'h', then there must be a smaller square-shaped area with all elements greater than or equal to 'h'. This contradicts the assumption that 'k' is the maximum size of the square-shaped area.

Thus, the algorithm correctly finds the largest square-shaped area in the matrix.

By the principle of mathematical induction, the algorithm is correct for all binary matrices of size $m \times n$.

Time Complexity analysis

The algorithm uses a 3-level nested loop to fill the DP matrix. The outermost loop iterates over the rows of the matrix, and the two inner loops iterate over the columns. The innermost loop has a maximum length of 'n', which is the number of columns in the matrix.

Within the innermost loop, the algorithm performs constant time operations like accessing matrix elements and updating DP values. Therefore, the time complexity of the innermost loop is $\Theta(1)$.

Since the outermost two loops have a combined length of 'mn', the time complexity of the algorithm is $\Theta(mn^2)$.

Space Complexity analysis

The algorithm uses a DP matrix of size 'm' x 'n' to store intermediate results. The size of the DP matrix is the same as the size of the input matrix, so the space complexity of the algorithm is $O(mn)$.

Additionally, the algorithm uses a constant amount of extra space to store variables like 'zero' and 'max_size'. The space required for these variables does not depend on the size of the input matrix, so it can be ignored in the asymptotic analysis.

Therefore, the space complexity of the algorithm is $O(mn)$.

Algorithm 5

Algorithm

Definition:

Let $DP[i][j]$ be the size of the largest square-shaped area whose bottom-right corner is at (i,j) .

Goal: $\max(DP[i][j])$ is the largest square in the matrix

Cases:

If $p[i][j] < h$, then $DP[i][j]$ is computed as follows:

If (i,j) is the top-left or top-right or bottom-left corner of a square whose height is greater than or equal to h , then $DP[i][j]$ is set to 1.

Otherwise, $DP[i][j]$ is set to $1 + \min(DP[i-1][j-1], DP[i-1][j], DP[i][j-1])$.

If $p[i][j] \geq h$, then $DP[i][j]$ is computed as $1 + \min(DP[i-1][j-1], DP[i-1][j], DP[i][j-1])$.

The Bellman equation for the recurrence relation is as follows:

$$\begin{aligned}
 &= 1 \text{ if } p[i][j] < h \text{ and } p[i][j+1] \geq h \text{ and } p[i+1][j] \geq h \text{ and } p[i+1][j+1] \geq h \\
 &= 1 \text{ if } p[i][j] < h \text{ and } p[i][j-1] \geq h \text{ and } p[i+1][j] \geq h \text{ and } p[i+1][j-1] \geq h \\
 DP[i][j] &= 1 \text{ if } p[i][j] < h \text{ and } p[i-1][j] \geq h \text{ and } p[i][j+1] \geq h \text{ and } p[i-1][j+1] \geq h \\
 &= 1 + \min(DP[i-1][j-1], DP[i-1][j], DP[i][j-1]), \text{ if } p[i][j] < h \text{ and } p[i-1][j-1] \geq h \text{ and } p[i][j-1] \geq h \\
 &\quad \text{and } p[i-1][j] \geq h \\
 &= 1 + \min(DP[i-1][j-1], DP[i-1][j], DP[i][j-1]), \text{ if } p[i][j] \geq h
 \end{aligned}$$

Correctness Proof

Base case:

The base case is when the size of the square-shaped area is 1×1 , i.e., $DP[0][0]$ or $DP[i][0]$ or $DP[0][j]$, where i and j are any indices such that $0 \leq i < m$ and $0 \leq j < n$.

The base case holds trivially since $DP[i][0]$ and $DP[0][j]$ are initialized to 0, and $DP[0][0]$ is set to 1 if $p[0][0] \geq h$, and 0 otherwise.

Inductive hypothesis:

Let $k \geq 2$ be the size of the square-shaped area. Assume that the algorithm correctly computes $DP[i][j]$ for all subproblems of size $k-1$, where i and j are any indices such that $k-1 \leq i < m$ and $k-1 \leq j < n$.

Inductive step:

We need to show that the algorithm correctly computes $DP[i][j]$ for a subproblem of size k , where $k \leq \min(m, n)$.

Let (i, j) be the bottom-right corner of the largest square-shaped area of size k whose height is greater than or equal to h .

If $p[i][j] < h$, then the square-shaped area cannot extend beyond cell (i, j) , i.e., it must have a smaller size, say k' . Therefore, we need to show that the algorithm correctly computes $DP[i][j]$ for a subproblem of size k' .

Since (i, j) is not the top-left corner of a square whose height is greater than or equal to h , we must have $DP[i][j] = 1 + \min(DP[i-1][j-1], DP[i-1][j], DP[i][j-1])$.

By the inductive hypothesis, we know that the algorithm correctly computes $DP[i-1][j-1]$, $DP[i-1][j]$, and $DP[i][j-1]$ for subproblems of size $k-1$.

Therefore, the algorithm computes $DP[i][j]$ correctly for a subproblem of size k when $p[i][j] < h$.

If $p[i][j] \geq h$, then the square-shaped area of size k whose height is greater than or equal to h must extend beyond cell (i, j) . Therefore, we need to show that the algorithm correctly computes $DP[i][j]$ for a subproblem of size $k-1$.

In this case, $DP[i][j] = 1 + \min(DP[i-1][j-1], DP[i-1][j], DP[i][j-1])$.

By the inductive hypothesis, we know that the algorithm correctly computes $DP[i-1][j-1]$, $DP[i-1][j]$, and $DP[i][j-1]$ for subproblems of size $k-1$.

Therefore, the algorithm computes $DP[i][j]$ correctly for a subproblem of size k when $p[i][j] \geq h$.

Since the inductive hypothesis holds for all subproblems of size $k-1$, and the algorithm correctly computes $DP[i][j]$ for a subproblem of size k using the recurrence relation, we can conclude that the algorithm correctly computes $DP[i][j]$ for all subproblems of size $k \leq \min(m, n)$ by induction.

Therefore, by induction, the algorithm is correct and computes the largest square-shaped area whose number of trees is greater than or equal to h and corner plots are of any size.

Time Complexity Analysis

The algorithm involves filling up a 2D DP matrix of size $m \times n$, where each entry is computed only once. The algorithm involves two nested loops that iterate over all the entries of the matrix

p, and for each entry, a constant amount of work is done to update the corresponding entry of the DP matrix.

Therefore, the total time complexity of the algorithm is proportional to the number of entries in the DP matrix, which is $m \times n$. Thus, the time complexity of the algorithm is $\Theta(mn)$.

Space Complexity Analysis

The algorithm involves creating a 2D DP matrix of size $m \times n$ to store the intermediate results. Each entry in the DP matrix requires constant space to store a single integer value. Therefore, the total space required to store the DP matrix is proportional to the number of entries, which is $m \times n$.

In addition to the DP matrix, the algorithm also uses a few constant amount of extra space to store the input matrix p, the integer value h, and some loop indices. However, this space is negligible compared to the space required for the DP matrix, and hence the overall space complexity of the algorithm remains $\Theta(mn)$.

Algorithm 6

Algorithm

Initialize max_size to 0 and boundary to an empty list.

For i in 1 to m:

 For j in 1 to n:

 for 1 to min(m-i, n-j)+1:

 for i to i+size-1:

 for j to j+size-1:

- Call countEnclosedPlotsAndMinTreeRequirement function to count the number of enclosed plots within the square

- If the number of enclosed plots is less than or equal to k:

- If the size of the square is greater than max_size, update max_size to the size of the square and update boundary to [i, j, i+size-1, j+size-1].

Return boundary.

Algorithm for the function countEnclosedPlotsAndMinTreeRequirement:

Initialize num_enclosed_plots to 0.

for i to i+s-1:

 for j to j+s-1:

 If the height of the current plot is less than h, increment num_enclosed_plots by 1.

Return num_enclosed_plots.

Correctness Proof

Base Case:

For the smallest input of $m = 1$ and $n = 1$, the algorithm will only check one square, which is the entire matrix, and will correctly return the boundary of this square if it contains at most k enclosed plots. Therefore, the base case holds.

Inductive Hypothesis:

Assume that for all input matrices with size (m', n') where $m' + n' < m + n$, the algorithm returns the correct output.

Inductive Step:

Consider an input matrix with size (m, n) , where $m + n > 2$. To apply the inductive hypothesis, we need to show that the algorithm correctly handles one square at a time, and that by checking all possible squares, it can find the largest square that meets the criteria.

Let's assume that the algorithm has checked all squares with a top-left corner at (i, j) where $i + j < k$, and has correctly found the largest square that meets the criteria. Now, let's consider a square with a top-left corner at (i', j') where $i' + j' = k$. This square has not been checked by the algorithm yet.

The algorithm will check all possible sizes of this square, starting from size 1 up to the maximum possible size, which is $\min(m - i', n - j') + 1$. For each size, it will count the number of enclosed plots using the `countEnclosedPlotsAndMinTreeRequirement` function. If the number of enclosed plots is less than or equal to k , and the size of the square is greater than the current maximum size, the algorithm will update the maximum size and the boundary.

Now, we need to show that the algorithm will eventually check the largest square with a top-left corner at (i', j') . Note that the algorithm will start by checking the smallest square with a top-left corner at (i', j') . If this square does not meet the criteria, the algorithm will move on to the next larger square with a top-left corner at (i', j') . This process will continue until the algorithm finds the largest square that meets the criteria.

Since the inductive hypothesis assumes that the algorithm correctly handles all squares with a top-left corner at (i, j) where $i + j < k$, and that it correctly finds the largest square that meets the criteria, we can conclude that the algorithm will correctly handle the square with a top-left corner at (i', j') and will eventually find the largest square that meets the criteria.

Therefore, by induction, we have shown that the `maximal_square` algorithm correctly returns the boundary of the largest square that can be formed with at most k enclosed land plots.

Time Complexity Analysis

The algorithm involves considering every possible sub-matrix of the input matrix p , which can be done by iterating over all possible pairs of row and column indices, and then iterating over all possible sizes of sub-matrices. For each sub-matrix, the algorithm then checks whether it is a square matrix with all its elements less than or equal to the integer value h . This requires checking every element of the sub-matrix, which takes $O(s^2)$ time, where s is the size of the sub-matrix.

Therefore, the total time complexity of the algorithm is proportional to the number of sub-matrices that need to be considered, which is $O(m^2n^2)$ multiplied by the time taken to check each sub-matrix, which is $O(s^2)$. The maximum size of any sub-matrix is $\min(m,n)$, and there are $O(mn)$ such sub-matrices, so the overall time complexity of the algorithm is $O(m^3n^3)$.

Space Complexity Analysis

The algorithm only uses a few constant amount of variables to store the input matrix p , the integer value h , and some loop indices. It does not create any additional data structures to store intermediate results or any other information.

Therefore, the overall space complexity of the algorithm remains constant and is independent of the size of the input matrix.

Algorithm 7

Algorithm

The optimal substructure property of the problem is that the size of the largest connected region starting at a given cell (i,j) can be computed recursively as the sum of the size of the current cell (i,j) plus the sizes of the largest connected regions starting at each of its neighboring cells, subject to the constraint that at most k cells with value less than h can be included in the region.

Let $S(i,j,k)$ be the size of the largest connected region starting at cell (i,j) with at most k cells with value less than h

$$S(i,j,k) = \max(1 + S(i-1, j-1, k - \text{count}), \\ S(i, j-1, k), \\ S(i-1, j, k)),$$

where count is the number of cells with value less than h in the connected region starting at cell (i,j) .

Correctness Proof

Base case:

$P(i, j, 0)$ holds trivially since there are no more "decreases" allowed and the size of the submatrix is simply 1 if the starting element satisfies the criteria, and 0 otherwise.

Inductive step:

We want to show that if $P(i', j', k')$ holds for all i', j', k' such that $0 \leq i' \leq i$, $0 \leq j' \leq j$, and $0 \leq k' \leq k$, then $P(i, j, k)$ also holds.

Suppose that we call $\text{find}(i, j, k)$. We first check if the result has already been computed and return it if so. Otherwise, we initialize ts to 0 and start to compute the maximum submatrix size starting at (i, j) and using at most k "decreases".

If the current element satisfies the criteria (i.e., it is greater than or equal to h or we have "decreases" remaining), we subtract 1 from k if the element is less than h . We then iterate over all possible submatrix sizes starting from 1 and going up to the maximum size that can fit within the remaining rows and columns of the matrix. For each submatrix size d , we compute the maximum size of a submatrix starting at $(i - 1, j - 1)$ and using at most $k - c(d)$ "decreases", where $c(d)$ is the number of elements in the current row that are less than h and are not already covered by the submatrix.

By the inductive hypothesis, we know that $P(i - 1, j' - 1, k - c(d))$ holds for all j' such that $j' - 1 < j$. Therefore, we can correctly compute the maximum size of a submatrix starting at $(i - 1, j - 1)$ and using at most $k - c(d)$ "decreases" for all valid submatrix sizes d .

We then update ts with the maximum submatrix size found and return ts . We also store the result in the `hash_map` dictionary to avoid redundant calculations.

Since we have correctly computed the maximum size of a submatrix starting at (i, j) and using at most k "decreases", $P(i, j, k)$ holds. Therefore, the algorithm maintains the desired property throughout its execution.

Finally, the algorithm loops over all elements in the matrix and finds the largest submatrix that satisfies the criteria. By the correctness of the `find` function, we know that the algorithm will correctly compute the largest submatrix that satisfies the criteria.

Time Complexity Analysis

The main loop of the algorithm iterates over all $m \times n$ elements of the input matrix. For each element, the `find` function is called, which can take up to $O(kn)$ time in the worst case. This is because we iterate over all submatrix sizes starting from 1 and going up to the maximum size that can fit within the remaining rows and columns of the matrix, which can be up to $O(n)$.

For each submatrix size, we need to compute the maximum submatrix size starting at the element $(i - 1, j - 1)$, which can take up to $O(kn)$ time in the worst case. This is because we need to iterate over all possible submatrix sizes, and for each submatrix size, we need to compute the maximum submatrix size starting at $(i - 1, j - 1)$ and using at most $k - c(d)$ "decreases", where $c(d)$ is the number of elements in the current row that are less than h and are not already covered by the submatrix. The maximum value of $c(d)$ is n , so the worst-case time complexity of computing the maximum submatrix size for each submatrix size is $O(kn)$. Therefore, the total time complexity of the algorithm is $\Theta(mnk)$.

Space Complexity Analysis

The main source of space usage in the algorithm is the hash map used to memoize the results of the find function. The size of the hash map can be up to $m \times n \times k$ in the worst case, because there are $m \times n$ possible starting positions for the find function, and each starting position can make up to k recursive calls with different parameters. Therefore, the worst-case space complexity of the hash map is $O(mnk)$.

In addition to the hash map, there are some local variables used within the find function, such as `start`, `ts`, `d`, and `temp`, that use constant space. There are also some variables used outside the find function, such as `r_x`, `r_y`, and `r_s`, that use constant space.

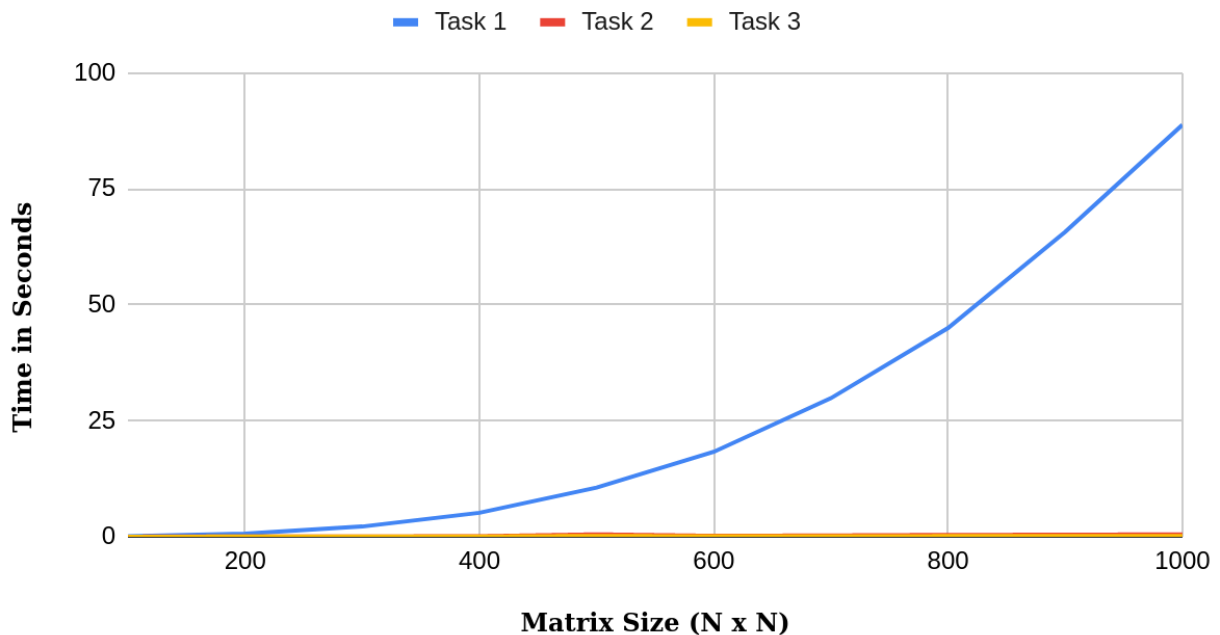
Therefore, the total space complexity of the algorithm is $\Theta(mnk)$.

Experimental Comparative Study

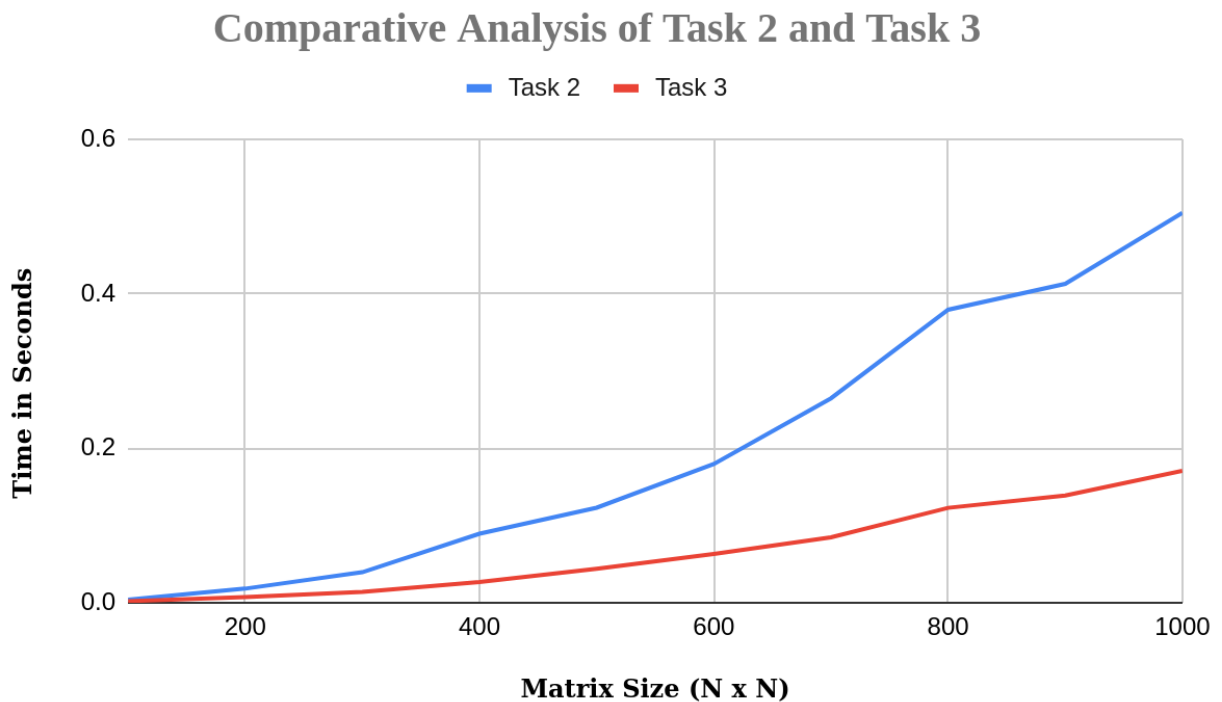
Note: The below figure shows the comparative study of Task 6, Task 7a and Task 7b. Since, Task 6 is designed with a $\Theta(m^3n^3)$ time Brute Force algorithm, comparative analysis with matrix of size ranging from 100 - 1000 would not be possible with the available computing resources. Thus analysis has been performed using the matrix of size ranging from 10 - 100.

Matrix Size (N x N) / Tasks	100	200	300	400	500	600	700	800	900	1000
1	0.0844	0.6711	2.1918	5.1603	10.6	18.3007	29.8662	45.0082	65.7278	88.802
2	0.004	0.0182	0.0394	0.0892	0.1229	0.1795	0.2644	0.379	0.4125	0.5042
3	0.002	0.0071	0.014	0.0268	0.0438	0.0631	0.0845	0.1227	0.1387	0.1705
4	0.0576	0.4794	1.6511	4.1023	7.0708	13.1424	21.0432	31.327	46.7229	62.927
5a	0.0075	0.0298	0.074	0.1289	0.1808	0.3154	0.3867	0.5326	0.7456	0.8081
5b	0.0045	0.0181	0.0418	0.0657	0.0928	0.142	0.1989	0.2931	0.3759	0.4331
7a	0.7101	3.4078	7.9522	14.6787	26.9838	37.0288	53.9816	71.7251	85.7657	104.9141
7b	0.4778	2.2778	4.9918	9.2516	17.0232	23.9493	33.7187	47.2893	54.6323	68.1827

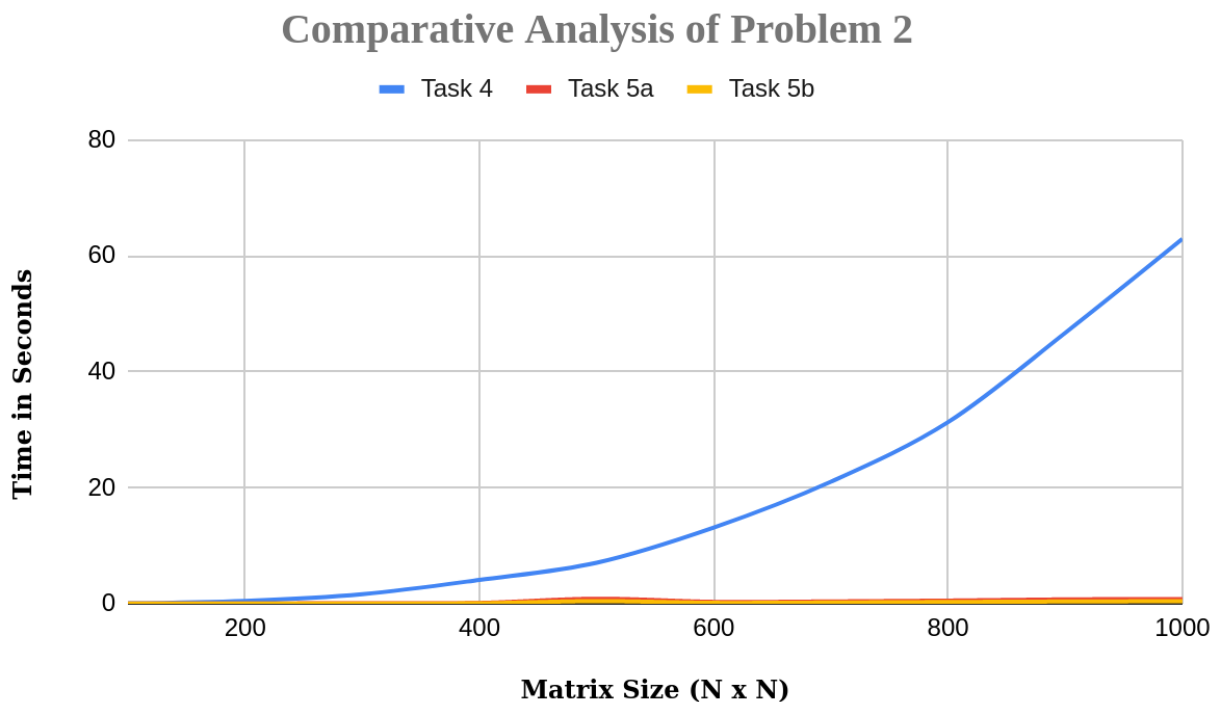
Comparative Analysis of Problem 1



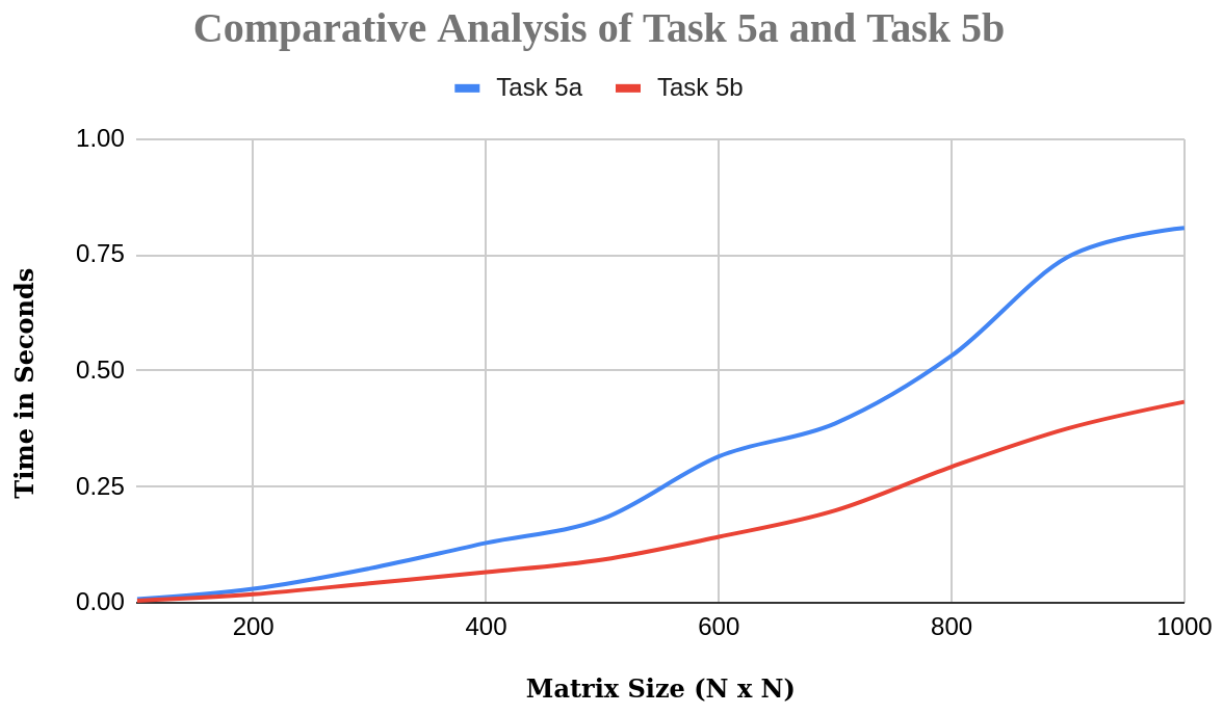
As we saw in the above chart the values of Task1 grossly overshadow the tasks 2 and 3 so we have added another graph below only to compare task 2 and 3.



In the above graph we see that task 2 increases in m^2n^2 time whereas task 3 increases in mn time



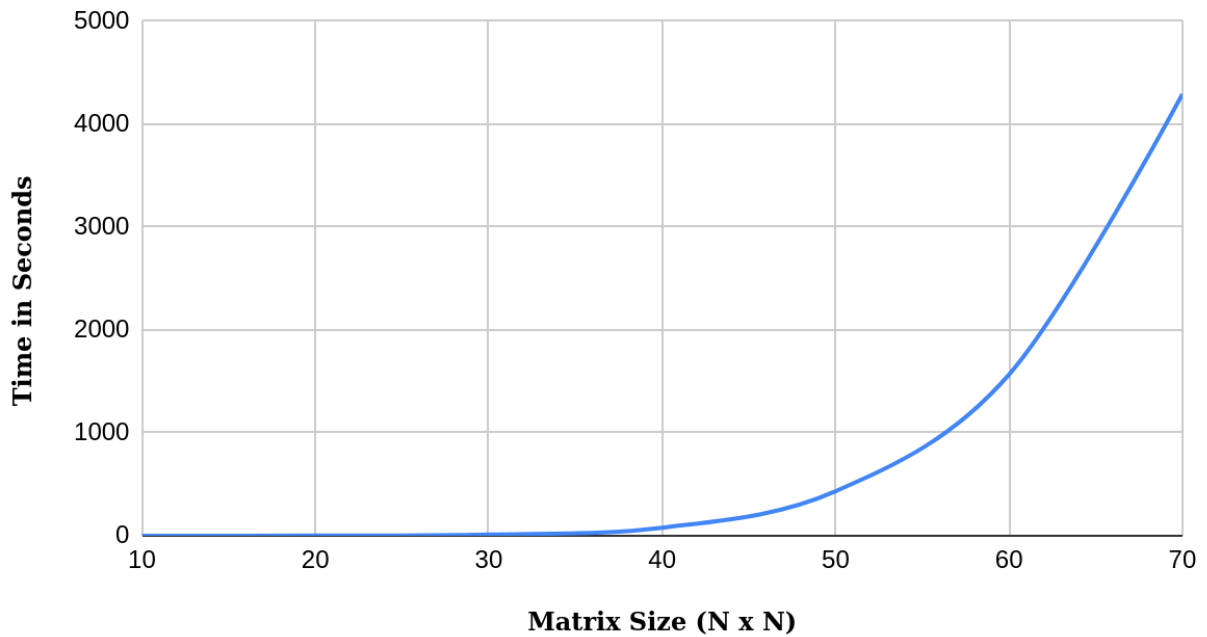
Task 4 is increasing in mn^2 and hence is overshadowing the Algorithm 5 graphs as they are in mn . So we've compared the two algorithm 5 approaches separately below



In the above graph task 5a grows at a higher rate and takes more time as the input size increases because each value is calculated multiple times as it can be left index, above index or diagonal value in a in $M \times N$ matrix which stores the cumulative sum till index i,j multiple times for a given i,j . Whereas in task 5b we are storing the values as and when we calculate it so the number of times each value is recalculated is linear.

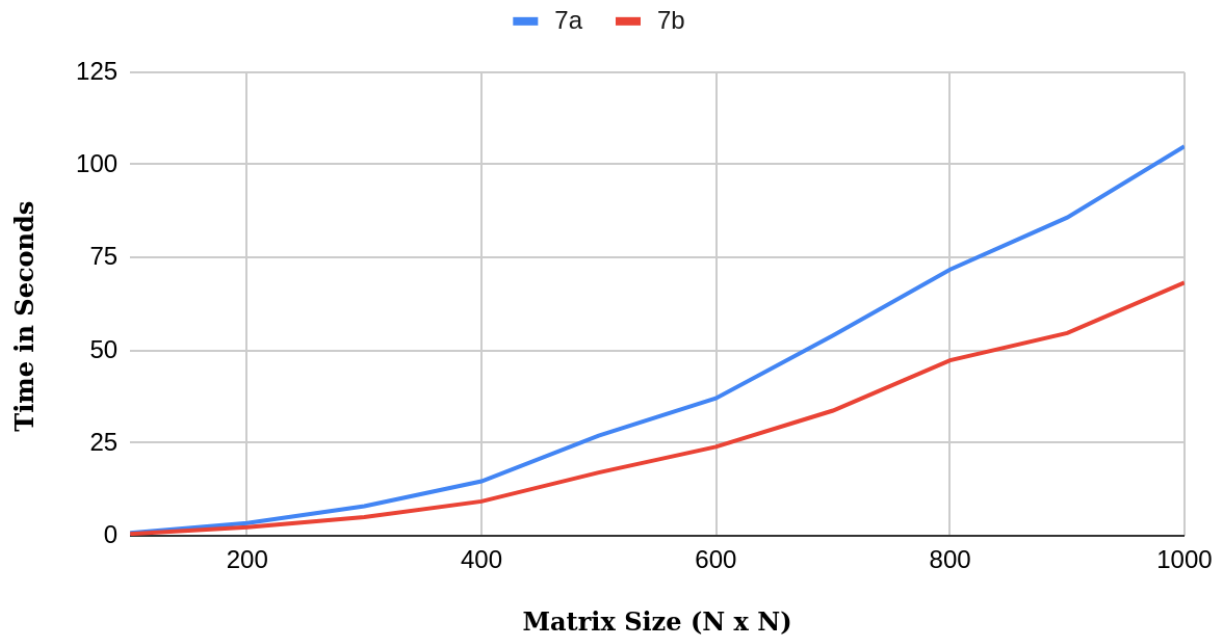
Matrix Size (N x N)	10	20	30	40	50	60	70
Task 6	0.0115	0.7929	11.234	77.2456	432.2467	1565.3365	4290.688

Analysis of Task 6



In the above graph we can observe that task 6 is a m^3n^3 algorithm and hence the execution time increases at a frightening pace and hence we have only run the execution for upto 70 x 70 input matrix. The k value used for the analysis is 15.

Comparative Analysis of Task 7a and Task 7b



The task 7 is a 3D dynamic programming algorithm and since Task 7a is a recursive implementation it is taking longer due to the usage of multi dimensional memory stacks to store the 3d matrix we are building when compared with Task 7b. The k value used for the analysis is 15.

Conclusion

This project is a comparative study which involves the designing and implementation of algorithms using multiple programming paradigms. The main aim is to understand the efficiency which dynamic programming offers in terms of time over brute force algorithms at the expense of space. The challenges we face in developing algorithms are introduced with the constraint in Problem 2 and Problem 3. This forces us to think of multiple base cases and involved parameterized equations clearly defining the goal and Bellman equation. For instance, where on one hand designing the solution as described in Task 7a requires us to think of careful usage of memoization which improves the time complexity and use of 3D DP in Task 7b with pre-populated base cases to support all the constraints. In problem 2 we are adding more checks to ensure that the corner values are also included in the answer and they can be used to increase the size of the square where trees can be planted. In that problem we also observe the different structures and approaches in which dynamic programming can be applied to solve a given problem such as memoization and bottom up approach. Overall the project is a good case study to understand the working and applications of dynamic programming in solving a problem in an efficient manner as compared to brute force or other approaches of designing an algorithm.